
🧪 Kubernetes DaemonSet Lab

(kOps-Managed Cluster | Scaling Nodes from 2 → 3)

📌 Lab Overview

This lab teaches how **DaemonSets work in Kubernetes** and how they behave when **cluster nodes are scaled using kOps**.

You will:

- Deploy and manage a DaemonSet
- Observe DaemonSet behavior across nodes
- Scale worker nodes using kOps
- See DaemonSet pods automatically adjust
- Practice real-world operational commands

🎯 Learning Objectives

By the end of this lab, you will be able to:

- Explain what a DaemonSet is and when to use it
- Deploy, view, update, and delete DaemonSets
- Debug DaemonSet pods
- Restrict DaemonSets to specific nodes
- Scale Kubernetes worker nodes using kOps
- Observe DaemonSet behavior when nodes are added

🧰 Prerequisites

- Kubernetes cluster created using **kOps**
- **2 worker nodes** initially
- kubectl and kops configured
- Basic knowledge of Pods and Nodes

Verify cluster access:

kubectl get nodes

📘 What Is a DaemonSet?

A **DaemonSet** ensures that **one pod runs on every node** (or selected nodes) in a cluster.

Common Use Cases

- Log collection (Fluentd, Filebeat)
- Monitoring agents (Node Exporter)
- Security agents
- Networking components

Key Characteristics

Feature	DaemonSet
Replicas	One per node
Auto-scales	Yes (with nodes)
Manual scaling	❌ Not allowed
Node-aware	✅ Yes

🔍 Viewing DaemonSets

List all DaemonSets in all namespaces

kubectl get daemonsets --all-namespaces

List DaemonSets in a specific namespace

kubectl get daemonsets -n <namespace>

Describe a DaemonSet (detailed info)

kubectl describe daemonset <name> -n <namespace>

Get pods managed by a DaemonSet

kubectl get pods -l <label-selector> -n <namespace>

🧪 Lab 1: Create a Basic DaemonSet

Step 1: Create a Namespace

kubectl create namespace daemonset-lab

Step 2: Create DaemonSet Manifest

Create the file:

vi daemonset-nginx.yaml

apiVersion: apps/v1
kind: DaemonSet
metadata:

```
  name: nginx-daemonset
  namespace: daemonset-lab
spec:
  selector:
    matchLabels:
      app: nginx-daemon
  template:
    metadata:
      labels:
        app: nginx-daemon
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
```

Step 3: Apply the DaemonSet

kubectl apply -f daemonset-nginx.yaml

Step 4: Verify DaemonSet

kubectl get daemonset -n daemonset-lab

Expected output:

```
NAME              DESIRED   CURRENT   READY
nginx-daemonset   2         2         2
```

❓ **Why DESIRED = 2?**

Because your cluster currently has **2 worker nodes**.

Step 5: Verify Pods per Node

kubectl get pods -n daemonset-lab -o wide

You should see:

- **2 pods**
- Each pod running on a **different node**

🧠 **Lab 2: Prove DaemonSet Is Node-Based**

Step 1: Try Scaling (Will Not Work)

kubectl scale daemonset nginx-daemonset --replicas=5 -n daemonset-lab

🔴 DaemonSets **ignore replicas**.

And got:

Error from server (NotFound): the server could not find the requested resource

🔍 **What's really happening?**

This is expected behavior because:

> 🚫 **DaemonSets do NOT support scaling with replicas**

Important Concept (Exam + Real World)

- kubectl scale works for:
    - Deployments
    - ReplicaSets
    - StatefulSets
- ❌ It does **not apply to DaemonSets**

That's why the API returns a **NotFound / unsupported resource error**.

💡 Kubernetes is protecting you from doing something that doesn't make sense for a DaemonSet.

🧠 **Key Rule to Remember**

> **You never scale a DaemonSet directly**
> You scale the **nodes**, and the DaemonSet follows automatically.

Step 2: Delete a DaemonSet Pod

kubectl delete pod <pod-name> -n daemonset-lab

Immediately check:

kubectl get pods -n daemonset-lab

✅ Pod is **recreated automatically** on the same node.

🧪 Lab 3: Run DaemonSet on Only One Node

Step 1: Label a Node

kubectl label node <node-name> disk=ssd

Verify labels:

kubectl get nodes --show-labels

Step 2: Update DaemonSet with Node Selector

Edit daemonset-nginx.yaml:

```
spec:
  selector:
    matchLabels:
      app: nginx-daemon
  template:
    metadata:
      labels:
        app: nginx-daemon
    spec:
      nodeSelector:
        disk: ssd
      containers:
      - name: nginx
        image: nginx:1.25
```

Apply changes:

kubectl apply -f daemonset-nginx.yaml

Expected Result

- DaemonSet runs **only on the labeled node**
- DESIRED = 1

⚙️ Creating, Updating & Managing DaemonSets

Edit a DaemonSet

kubectl edit daemonset <name> -n <namespace>

Delete a DaemonSet

kubectl delete daemonset <name> -n <namespace>

🔄 Rolling Updates & Management

Restart DaemonSet pods

kubectl rollout restart daemonset <name> -n <namespace>

Check rollout status

kubectl rollout status daemonset <name> -n <namespace>

Pause / Resume rollout

kubectl rollout pause daemonset <name> -n <namespace>
kubectl rollout resume daemonset <name> -n <namespace>

📄 YAML Export & Debugging

Export DaemonSet YAML

kubectl get daemonset <name> -n <namespace> -o yaml

View logs

kubectl logs <pod-name> -n <namespace>

Exec into a pod

kubectl exec -it <pod-name> -n <namespace> -- /bin/sh

🚀 Lab 4: Scale Nodes from 2 → 3 Using kOps

Step 1: List Instance Groups

kops get ig

Example output:

```
NAME                ROLE   MACHINETYPE  MIN  MAX  ZONES
nodes-us-west-2a    Node   t3.medium    2    2    us-west-2a
master-us-west-2a   Master t3.medium    1    1    us-west-2a
```

⚠️ Note the **exact Node instance group name**.

Step 2: Edit the Node Instance Group

kops edit ig nodes-us-west-2a

Change:

spec:
  minSize: 2
  maxSize: 2

To:

spec:
  minSize: 3
  maxSize: 3

Save and exit.

Step 3: Apply the Changes

kops update cluster --yes
kops rolling-update cluster --yes

Step 4: Verify New Node

kubectl get nodes

Expected:

3 worker nodes in Ready state

🔥 Observe DaemonSet Auto-Scaling

kubectl get daemonset -n daemonset-lab

Expected:

DESIRED CURRENT READY
3 3 3

🎉 A **new DaemonSet pod** is automatically created on the new node.

🧠 Key Takeaways

- DaemonSets run **one pod per node**
- They **automatically adapt** to node changes
- kOps scales nodes via **Instance Groups**
- DaemonSets are ideal for **cluster-level services**

🧹 Cleanup (Optional)

kubectl delete namespace daemonset-lab

🚀 Bonus Exercises

- Add **tolerations** to run on master nodes
- Deploy **node-exporter** as a DaemonSet
- Scale back from **3 → 2 nodes**
- Add **resource limits** to DaemonSet pods
